Object Oriented Programming(C++)
(CSE 1211)

Friend Function

Md. Mahfuzur Rahman
Lecturer,
Prime University

Friend Class in C++

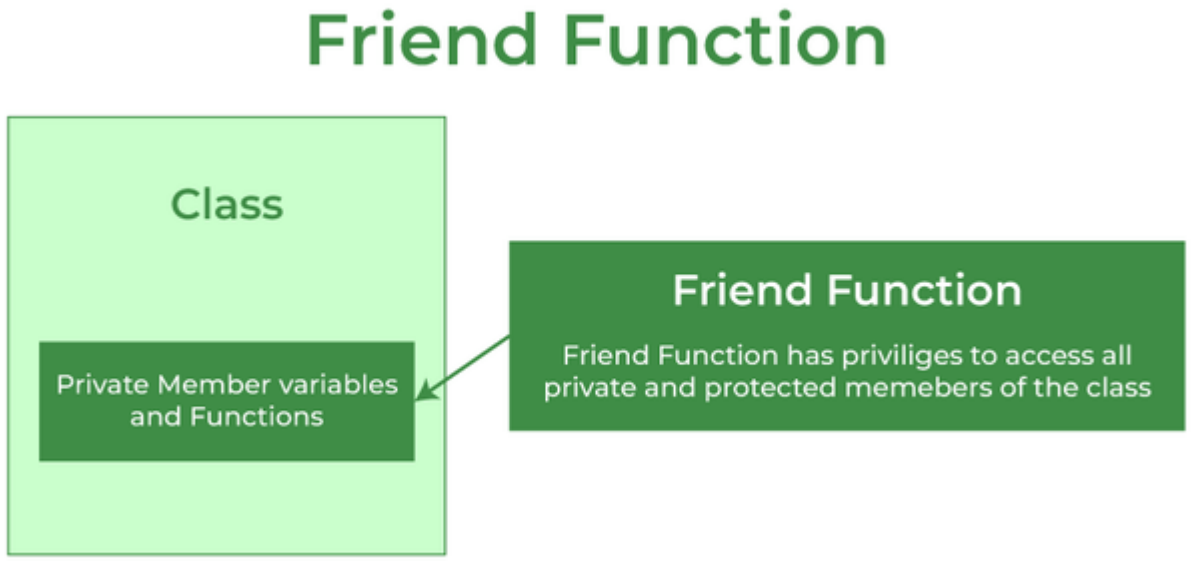
A **friend class** can access private and protected members of other classes in which it is declared as a friend. It is sometimes useful to allow a particular class to access private and protected members of other classes. Remember one thing, Friendship is not mutual. If class A is a friend of B, then B doesn't become a friend of A automatically.

A friend function can be:

- A global function
- A member function of another class

Friend Class in C++

friend function can be granted special access to private and protected members of a class in C++. They are not the member functions of the class but can access and manipulate the private and protected members of that class for they are declared as friends.



C++ Friend function

- If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.
- By using the keyword friend compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
   friend data_type function_name(argument/s);          // syntax of friend function.
};
```

In the above declaration, the friend function is preceded by the keyword friend. The function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword **friend or scope resolution operator**.

Friend Function

```cpp
#include<iostream>
using namespace std;

class A
{
private:
    int id = 101;
    string name = "prime";

public:
    friend class B;
};

class B
{
public:
    void display(A ob)
    {
        cout << ob.id << endl;
        cout << ob.name << endl;
    }
};

int main()
{
    A ob1;
    B ob2;
    ob2.display(ob1);
}
```

C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

```
#include <iostream>

using namespace std;

class A
{
   int x =5;
   friend class B;          // friend class.
};
class B
{
  public:
    void display(A &a)
    {
        cout<<"value of x is : "<<a.x;
    }
};

int main()
{
    A a;
    B b;
    b.display(a);
    return 0;
}
```

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

Advantages of Friend Functions

- Following is the list of advantages of friend functions in C++:
- A friend function is able to access members without the need of inheriting the class.
- The friend function acts as a bridge between two classes by accessing their private data.
- It can be used to increase the versatility of overloaded operators.
- It can be declared either in the public or private or protected part of the class.

Demerits of Friend Functions

- The friend function provides us with a way to access private data, but it also has its demerits:
- Friend functions have access to private members of a class from outside the class which violates the law of data hiding.
- Friend functions cannot do any run-time polymorphism in their members.
- Declaring too many functions or external classes as friends with access to a class’s private or protected data reduces the effectiveness of encapsulation. This compromises one of the core principles of object-oriented programming.
- Friendship is not inherited. In layman's terms, if a base class has a friend function, then the function doesn’t become a friend of the derived class(es).

Friend Function

```cpp
#include<iostream>
using namespace std;

class A
{
private:
    int id = 101;
    string name = "prime";

public:
    friend class B;
};

class B
{
public:
    void display(A ob)
    {
        cout << ob.id << endl;
        cout << ob.name << endl;
    }
};

int main()
{
    A ob1;
    B ob2;
    ob2.display(ob1);
}
```

C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

```
#include <iostream>

using namespace std;

class A
{
   int x =5;
   friend class B;          // friend class.
};
class B
{
  public:
    void display(A &a)
    {
        cout<<"value of x is : "<<a.x;
    }
};

int main()
{
    A a;
    B b;
    b.display(a);
    return 0;
}
```

C++ Friend function

- If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.
- By using the keyword friend compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
   friend data_type function_name(argument/s);          // syntax of friend function.
};
```

In the above declaration, the friend function is preceded by the keyword friend. The function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword **friend or scope resolution operator**.

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

C++ friend function Example

```
#include <iostream>
using namespace std;
class Box
{
   private:
      int length;
   public:
      Box(): length(0) { }
      friend int printLength(Box); //friend function
};
int printLength(Box b)
{
   b.length += 10;
    return b.length;
}
int main()
{
    Box b;
    cout<<"Length of box: "<< printLength(b)<<endl;
    return 0;
}
```

Thank You